endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 12

AGENDA CURS 12

1. Design Patterns – de ce?
2. Principalele design patterns
3. Singleton
4. Builder
5. Factory method
6. Abstract factory
7. Observer
8. Adapter

endava

DESIGN PATTERNS – DE CE?

Someone has already solved your problems.

endava

DESIGN PATTERNS – DE CE?

Source: Head First Design Patterns, 2nd Edition

DESIGN PATTERNS – DE CE?

The one constant in software development?

CHANGE

endava

PRINCIPALELE DESIGN PATTERNS

Categorie	Descriere scurta	Exemple de Design Patterns
Creational	Controleaza **crearea obiectelor**, separand procesul de instantiere de codul client	- Singleton - Factory Method - Abstract Factory - Builder - Prototype
Structural	Definesc **relatii intre clase/obiecte** pentru a forma structuri mai mari	- Adapter - Decorator - Proxy - Composite - Facade
Behavioral	Se ocupa cu **comunicarea intre obiecte** si comportamente dinamice	- Observer - Strategy - Command - State - Iterator - Template Method - Chain of Responsibility - Visitor

SINGLETON PATTERN

Source: Head First Design Patterns, 2nd Edition

SINGLETON PATTERN

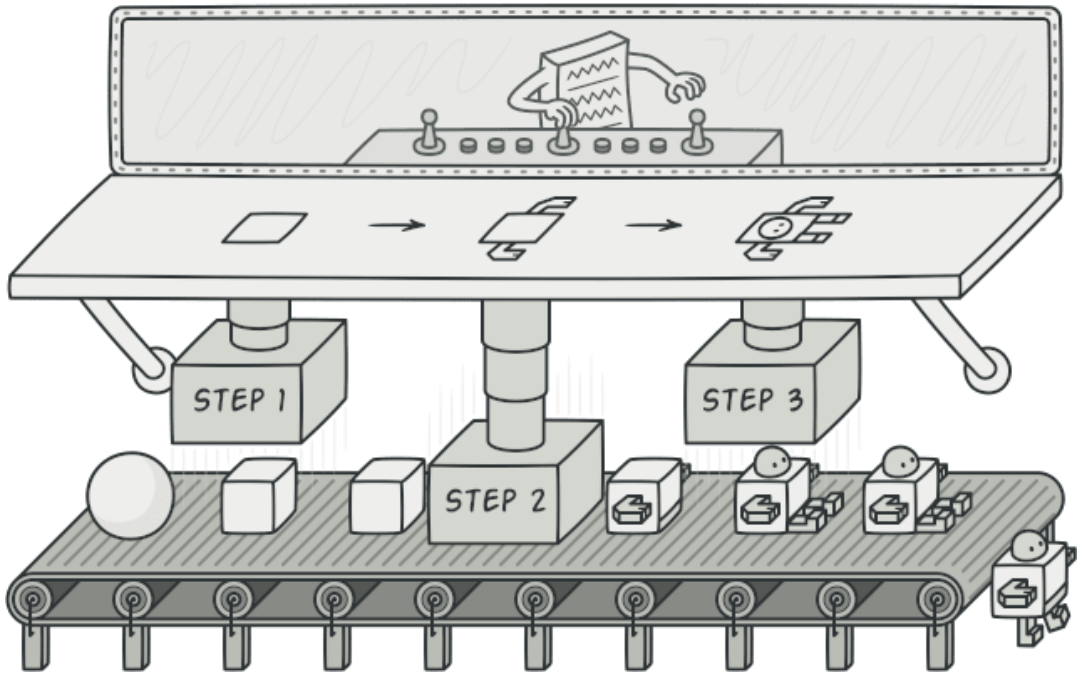
- Singleton este un design pattern creational care asigura ca o clasa are **o singura instanta** si ofera un **punct global de acces** la aceasta instanta.
- Evita crearea multiplelor instante ale unei clase. Crearea mai multor instante pentru anumite clase poate duce la comportament incorrect al aplicatiei, consum mare de resurse, rezultate inconsistente.
- Cand se foloseste:
  - Managementul resurselor partajate: conexiuni la baza de date, thread pools, caches, configurari diverse
  - Coordonarea cross-cutting concers: logging, file managers
  - Managementul starii: user session, user preferences
- Avantaje:
  - Control total asupra instantei unice.
  - Consum redus de resurse (in unele cazuri).
- Dezavantaje:
  - Poate cauza probleme in testare (dificil de simulat sau inlocuit).
  - Poate incalca principiul responsabilitatii unice daca este folosit excesiv.

SINGLETON PATTERN

- Tipuri de implementari:
  - Eager initialization
  - Lazy initialization
  - Thread-safe singleton
  - Double-checked locking
  - **Bill Pugh singleton**
  - **Enum singleton**
  - Static block initialization

endava

BUILDER PATTERN

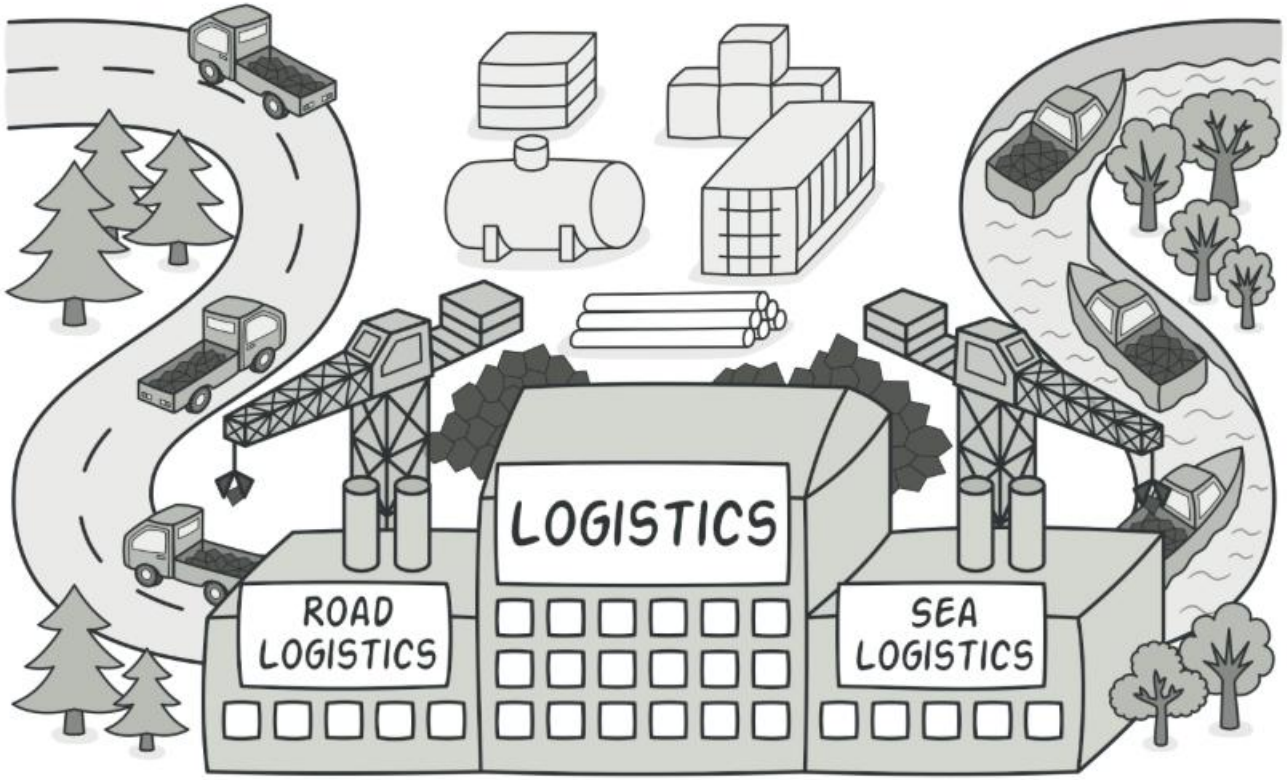


Source: https://refactoring.guru/design-patterns/builder

BUILDER PATTERN

- **Builder** este un design pattern creational folosit pentru a construi obiecte complexe pas cu pas, permitand o mai buna lizibilitate si flexibilitate fata de constructorii clasici.
- Cand se foloseste:
  - Obiectele au multi parametri, unii optionali.
  - Vrei un cod clar si usor de intretinut.
  - Vrei sa eviti supraincarcarea constructorilor
- Avantaje:
  - Cod clar si fluent.
  - Permite configurarea flexibila a obiectelor.
  - Evita constructorii lungi si greu de inteles.
  - Poate asigura imutabilitatea obiectului.
- Dezavantaje:
  - Cod mai lung.
  - Ușor prea complicat pentru obiecte simple.

FACTORY METHOD PATTERN

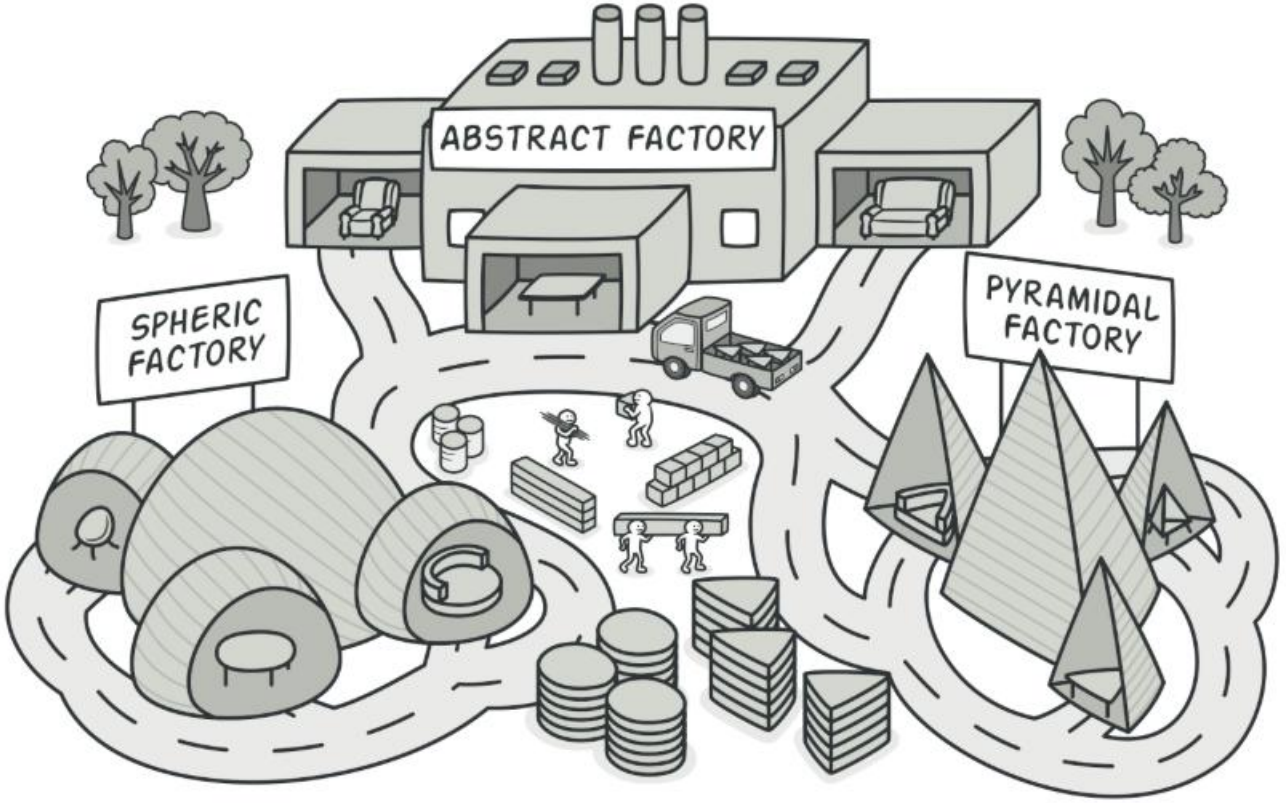


Source: https://refactoring.guru/design-patterns/factory-method

FACTORY METHOD PATTERN

- **Factory** este un design pattern creational folosit pentru a crea obiecte fara a expune logica instantiarii clientului. Obiectele sunt create printr-o metoda separata numita *factory method*.
- Cand se foloseste:
  - Cand nu stii dinainte ce tip exact de obiect trebuie creat.
  - Cand vrei sa ascunzi detaliile de creare a obiectelor.
  - Cand ai o familie de clase care implementeaza aceeasi interfata sau clasa abstracta.
- Avantaje:
  - Codul client nu este afectat de schimbarile in clasele concrete.
  - Permite introducerea usoara a unor noi tipuri de obiecte.
  - Respecta principiul Open/Closed (deschis pentru extensie, inchis pentru modificare).
- Dezavantaje:
  - Poate creste numarul de clase.
  - Logica din factory poate deveni complicata daca numarul de tipuri creste.

ABSTRACT FACTORY PATTERN



Source: https://refactoring.guru/design-patterns/abstract-factory

ABSTRACT FACTORY PATTERN

- **Abstract Factory** este un design pattern creational care ofera o interfata pentru crearea familiilor de obiecte inrudite sau dependente, fara a specifica clasele concrete.
- Cand se foloseste:
  - Cand aplicatia trebuie sa foloseasca familii de obiecte legate.
  - Cand vrei sa izolezi codul client de detalii concrete de creare.
  - Cand obiectele trebuie sa functioneze impreuna corect (ex: teme grafice, drivere, etc).
- Avantaje:
  - Codul client nu este afectat de schimbarile in clasele concrete.
  - Permite introducerea usoara a unor noi tipuri de obiecte.
  - Respecta principiul Open/Closed (deschis pentru extensie, inchis pentru modificare).
- Dezavantaje:
  - Poate creste numarul de clase.
  - Logica din factory poate deveni complicata daca numarul de tipuri creste.

ABSTRACT FACTORY PATTERN

Caracteristica	Factory Method	Abstract Factory
Tip de obiecte produse	Un singur tip de obiect	Mai multe obiecte (familii)
Numar de metode factory	De obicei una	Mai multe metode (pentru fiecare tip de obiect)
Exemplu din cod	`getConnection()`	`createButton()`, `createCheckbox()`
Scop principal	Delegare instantiere	Gruparea de obiecte compatibile

endava

OBSERVER PATTERN

Source: Head First Design Patterns, 2nd Edition

OBSERVER PATTERN

- **Observer** este un design pattern comportamental care defineste o relatie de tip **publish/subscribe** intre obiecte. Cand un obiect (subject) isi schimba starea, notifica automat toate obiectele dependente (observatori).
- Cand se foloseste:
  - Cand ai mai multe obiecte care trebuie actualizate automat cand unul se schimba.
  - In implementari de tip: UI events, notificari, fluxuri de date, etc.
- Avantaje:
  - Promoveaza loose coupling intre subiect si observatori.
  - Usor de extins cu noi observatori.
  - Comunicare automata si dinamica intre obiecte.
- Dezavantaje:
  - Notificarea tuturor observatorilor poate deveni costisitoare.
  - Dificil de investigat cand sunt multe notificari si dependente inlantuite.

ADAPTER PATTERN

Source: Head First Design Patterns, 2nd Edition

ADAPTER PATTERN

- **Adapter** este un design pattern structural care permite ca doua clase incompatibile sa colaboreze, facand interfata uneia compatibila cu cealalta.
- Cand se foloseste:
  - Cand vrei sa folosesti o clasa existenta, dar interfata ei nu se potriveste cu ce asteapta codul client.
  - Cand vrei sa integrezi cod vechi (legacy) intr-un sistem nou.
- Avantaje:
  - Permite compatibilitate intre clase cu interfete diferite.
  - Usor de integrat cod legacy in sisteme moderne.
  - Respecta principiul "open/closed" (nu modifici codul existent).
- Dezavantaje:
  - Poate introduce complexitate daca sunt prea multe adaptari.
  - Nu rezolva probleme de incompatibilitate de comportament, doar de interfata.

Q&A

endava

THANK YOU

DANIELA SPILCĂ